

Custom Comparator Practice —

Answers

Worked solutions for every question in `Comparator_Practice_Questions.docx`, in the same order and numbering. Default language is JavaScript (`Array.prototype.sort compareFn`); Section 5 uses the language named in each question. See `Comparator_Theory.docx` for the underlying concepts referenced here (stability, transitivity, `key=` vs `comparator`, etc.).

1. Warm-up: Basic Comparator Mechanics

1. Sort an array of integers in descending order using a custom comparator.

```
arr.sort((a, b) => b - a);
```

2. Sort strings by length, shortest first; break ties alphabetically.

```
arr.sort((a, b) => a.length - b.length || a.localeCompare(b));
```

3. Sort strings by length, longest first; break ties by reverse alphabetical order.

```
arr.sort((a, b) => b.length - a.length || b.localeCompare(a));
```

4. Even numbers before odd numbers, each group ascending.

```
arr.sort((a, b) => (Math.abs(a % 2) - Math.abs(b % 2)) || (a - b));
```

`Math.abs` handles negative operands, since JS's `%` can return `-1` for odd negatives.

5. Sort numbers by absolute value, ascending.

```
arr.sort((a, b) => Math.abs(a) - Math.abs(b));
```

6. Negative numbers first (descending), then positive numbers (ascending).

```
arr.sort((a, b) => {  
  if (a < 0 && b < 0) return b - a;  
  if (a >= 0 && b >= 0) return a - b;  
  return a < 0 ? -1 : 1;  
});
```

7. Sort strings by vowel count, descending.

```
const vowels = s => (s.match(/[aeiou]/gi) || []).length;  
arr.sort((a, b) => vowels(b) - vowels(a));
```

8. Sort words by last character, then alphabetically for ties.

```
arr.sort((a, b) =>
  a[a.length - 1].localeCompare(b[b.length - 1]) || a.localeCompare(b)
);
```

2. Sorting Arrays of Objects

9. Sort Person objects ({name, age}) by age ascending.

```
people.sort((a, b) => a.age - b.age);
```

10. Age descending; ties by name ascending.

```
people.sort((a, b) => b.age - a.age || a.name.localeCompare(b.name));
```

11. Employee objects: department asc, then salary desc, then name asc.

```
employees.sort((a, b) =>
  a.department.localeCompare(b.department) ||
  b.salary - a.salary ||
  a.name.localeCompare(b.name)
);
```

12. Product objects: rating descending; equal ratings by price ascending.

```
products.sort((a, b) => b.rating - a.rating || a.price - b.price);
```

13. Student objects by GPA descending; null/undefined GPA sorts last.

```
students.sort((a, b) => {
  const aNull = a.gpa == null, bNull = b.gpa == null;
  if (aNull && bNull) return 0;
  if (aNull) return 1;
  if (bNull) return -1;
  return b.gpa - a.gpa;
});
```

14. Order objects by orderDate, most recent first (ISO string).

```
orders.sort((a, b) => b.orderDate.localeCompare(a.orderDate));
```

ISO 8601 strings sort lexicographically in date order, so `localeCompare` (or plain string comparison) is enough; `new Date(b.orderDate) - new Date(a.orderDate)` also works.

15. Version objects ({major, minor, patch}) in proper semver order.

```
versions.sort((a, b) =>
  a.major - b.major || a.minor - b.minor || a.patch - b.patch
);
```

Numeric field comparison avoids the classic bug of treating versions as strings, where "1.10.0" would incorrectly sort before "1.9.0".

16. Task objects: priority descending, then deadline ascending for ties.

```
tasks.sort((a, b) =>
  b.priority - a.priority || new Date(a.deadline) - new Date(b.deadline)
);
```

17. Generic comparator factory: sort by a dynamic field name and direction.

```
function byField(field, dir = 1) {
  return (a, b) => {
    if (a[field] < b[field]) return -1 * dir;
    if (a[field] > b[field]) return 1 * dir;
    return 0;
  };
}
items.sort(byField('price', -1)); // descending by price
```

18. Custom, non-alphabetical priority order for a categorical field via a rank map.

```
const rank = { urgent: 0, high: 1, medium: 2, low: 3 };
items.sort((a, b) => rank[a.status] - rank[b.status]);
```

3. Sorting Arrays of Arrays (Pairs / Intervals)

19. [start, end] intervals: sort by start ascending, ties by end ascending.

```
intervals.sort((a, b) => a[0] - b[0] || a[1] - b[1]);
```

20. Merge overlapping intervals.

```
function merge(intervals) {
  intervals.sort((a, b) => a[0] - b[0]);
  const result = [];
  for (const [start, end] of intervals) {
    const last = result[result.length - 1];
    if (last && start <= last[1]) {
      last[1] = Math.max(last[1], end);
    } else {
      result.push([start, end]);
    }
  }
  return result;
}
```

21. [x, y] points sorted by distance from the origin, ascending.

```
points.sort((a, b) => (a[0]**2 + a[1]**2) - (b[0]**2 + b[1]**2));
```

Comparing squared distances avoids an unnecessary sqrt call per comparison.

22. [id, score] pairs: score descending, ties by id ascending.

```
pairs.sort((a, b) => b[1] - a[1] || a[0] - b[0]);
```

23. Meeting intervals: can a person attend all meetings (no overlaps)?

```
function canAttendAll(intervals) {  
  intervals.sort((a, b) => a[0] - b[0]);  
  for (let i = 1; i < intervals.length; i++) {  
    if (intervals[i][0] < intervals[i - 1][1]) return false;  
  }  
  return true;  
}
```

24. [name, votes] pairs: votes descending, ties by name ascending.

```
pairs.sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]));
```

25. Triplets [x, y, z]: sort by z asc, then x asc, then y desc.

```
triplets.sort((a, b) => a[2] - b[2] || a[0] - b[0] || b[1] - a[1]);
```

26. [word, frequency] pairs: frequency descending, ties alphabetical.

```
pairs.sort((a, b) => b[1] - a[1] || a[0].localeCompare(b[0]));
```

4. Sorting 2D Arrays / Matrices

27. Sort matrix rows by row sum, ascending.

```
const sum = row => row.reduce((s, v) => s + v, 0);  
matrix.sort((r1, r2) => sum(r1) - sum(r2));
```

28. Sort matrix rows by a specific column index, descending.

```
const col = 2;  
matrix.sort((r1, r2) => r2[col] - r1[col]);
```

29. Sort by column 0 ascending; ties by column 1 descending.

```
matrix.sort((r1, r2) => r1[0] - r2[0] || r2[1] - r1[1]);
```

30. Sort rows by average score, descending.

```
const avg = row => row.reduce((s, v) => s + v, 0) / row.length;  
matrix.sort((r1, r2) => avg(r2) - avg(r1));
```

31. Binary matrix: sort rows by count of 1s ascending, stable on ties.

```
matrix.sort((r1, r2) => {  
  const ones = row => row.reduce((c, v) => c + v, 0);  
  return ones(r1) - ones(r2);  
});
```

No extra tiebreaker is needed for stability: `Array.prototype.sort` has been a stable sort since ES2019, so rows with equal counts keep their original relative order automatically.

32. Cell coordinates [row, col]: sort row-major, then column-major.

```
// row-major (top-to-bottom, then left-to-right)
cells.sort((a, b) => a[0] - b[0] || a[1] - b[1]);

// column-major (left-to-right, then top-to-bottom)
cells.sort((a, b) => a[1] - b[1] || a[0] - b[0]);
```

33. Sort the columns of a matrix by column sum, ascending.

```
function sortColumnsBySum(matrix) {
  const numCols = matrix[0].length;
  const colSums = Array.from({ length: numCols }, (_, c) =>
    matrix.reduce((s, row) => s + row[c], 0)
  );
  const order = [...Array(numCols).keys()].sort((a, b) => colSums[a] - colSums[b]);
  return matrix.map(row => order.map(c => row[c]));
}
```

Sort the column indices by their aggregate, then rebuild every row using that index order — you can't sort columns directly since a JS array is row-major.

34. Sort a list of matrices by trace (sum of the diagonal), ascending.

```
const trace = m => m.reduce((s, row, i) => s + row[i], 0);
matrices.sort((m1, m2) => trace(m1) - trace(m2));
```

5. Comparator Objects / Comparable Interfaces (Java & Python Style)

35. Java: `Comparator<Employee>` as a separate class, sorting by salary descending.

```
class BySalaryDesc implements Comparator<Employee> {
  @Override
  public int compare(Employee a, Employee b) {
    return Double.compare(b.getSalary(), a.getSalary());
  }
}

employees.sort(new BySalaryDesc());
```

36. Java: `Comparator.comparing().thenComparing()` three-field chain.

```
employees.sort(
  Comparator.comparing(Employee::getDepartment)
    .thenComparing(Comparator.comparingDouble(Employee::getSalary).reversed())
    .thenComparing(Employee::getName)
);
```

37. Java: implement `Comparable<T>`, sort with `Collections.sort` and no external comparator.

```

class Employee implements Comparable<Employee> {
    double salary;
    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}
Collections.sort(employeeList); // uses compareTo automatically

```

38. Java: `Comparator.comparingInt(...).reversed()` vs manual subtraction overflow.

```

// Safe: uses Integer.compare internally, no overflow
list.sort(Comparator.comparingInt((Item i) -> i.value).reversed());

// Risky: a - b overflows if a and b are near Integer.MIN_VALUE/MAX_VALUE,
// which can silently flip the sign and corrupt the sort.
list.sort((a, b) -> b.value - a.value); // avoid for untrusted ranges

```

`Comparator.comparingInt/comparingLong/comparingDouble` always use the correct `compare()` method internally, so prefer them over manual subtraction whenever the value range isn't tightly controlled.

39. Python: `key=` with a lambda for multi-field sorting.

```

people.sort(key=lambda p: (-p.age, p.name))

```

Negating age flips it to descending while name stays ascending, all in one tuple key — this avoids `cmp_to_key` entirely because the whole ordering reduces to a single sortable tuple.

40. Python: `functools.cmp_to_key` — arrange numbers to form the largest possible number.

```

from functools import cmp_to_key

def largest_number(nums):
    strs = list(map(str, nums))
    strs.sort(key=cmp_to_key(lambda a, b: (b + a > a + b) - (b + a < a + b)))
    result = ''.join(strs)
    return '0' if result[0] == '0' else result

```

This ordering ("does `a+b` or `b+a` make the bigger number") can't be expressed as a per-element key, which is exactly when `cmp_to_key` earns its keep.

41. Python: `__lt__` and `__eq__` so instances sort directly with `sorted()`.

```

class Employee:
    def __init__(self, name, salary):
        self.name, self.salary = name, salary
    def __lt__(self, other):
        return self.salary < other.salary
    def __eq__(self, other):
        return self.salary == other.salary

```

```
sorted(employees) # uses __lt__ directly, no key or comparator needed
```

42. C++: custom comparator for vector<pair>, second ascending then first descending.

```
std::sort(v.begin(), v.end(), [](const auto& a, const auto& b) {  
    if (a.second != b.second) return a.second < b.second;  
    return a.first > b.first;  
});
```

6. Classic Interview Problems Built Around Custom Comparators

43. Largest Number.

```
function largestNumber(nums) {  
    const strs = nums.map(String);  
    strs.sort((a, b) => (b + a).localeCompare(a + b));  
    const result = strs.join("");  
    return result[0] === '0' ? '0' : result;  
}
```

44. Meeting Rooms II — minimum rooms required.

```
function minMeetingRooms(intervals) {  
    const starts = intervals.map(i => i[0]).sort((a, b) => a - b);  
    const ends = intervals.map(i => i[1]).sort((a, b) => a - b);  
    let rooms = 0, maxRooms = 0, s = 0, e = 0;  
    while (s < starts.length) {  
        if (starts[s] < ends[e]) { rooms++; s++; }  
        else { rooms--; e++; }  
        maxRooms = Math.max(maxRooms, rooms);  
    }  
    return maxRooms;  
}
```

45. Merge K Sorted Lists — min-heap on node value.

```
// Node: { val, next }. Using a simple array-backed min-heap keyed by node.val.  
function mergeKLists(lists) {  
    const heap = new MinHeap((a, b) => a.val - b.val);  
    for (const node of lists) if (node) heap.push(node);  
    const dummy = { next: null };  
    let tail = dummy;  
    while (heap.size() > 0) {  
        const node = heap.pop();  
        tail.next = node;  
        tail = node;  
        if (node.next) heap.push(node.next);  
    }  
    return dummy.next;  
}
```

MinHeap is a generic binary heap taking a comparator, as built in Section 7.

46. K Closest Points to Origin.

```
function kClosest(points, k) {  
  return points  
    .slice()  
    .sort((a, b) => (a[0]**2 + a[1]**2) - (b[0]**2 + b[1]**2))  
    .slice(0, k);  
}
```

A full sort is $O(n \log n)$; for very large n with small k , a max-heap of size k (see Section 7) does it in $O(n \log k)$.

47. Top K Frequent Elements.

```
function topKFrequent(nums, k) {  
  const freq = new Map();  
  for (const n of nums) freq.set(n, (freq.get(n) || 0) + 1);  
  return [...freq.keys()]  
    .sort((a, b) => freq.get(b) - freq.get(a))  
    .slice(0, k);  
}
```

48. Custom Sort String.

```
function customSortString(order, s) {  
  const rank = new Map([...order].map((ch, i) => [ch, i]));  
  return [...s]  
    .sort((a, b) => (rank.get(a) ?? 999) - (rank.get(b) ?? 999))  
    .join("");  
}
```

49. Task Scheduler.

```
function leastInterval(tasks, n) {  
  const freq = new Map();  
  for (const t of tasks) freq.set(t, (freq.get(t) || 0) + 1);  
  const counts = [...freq.values()].sort((a, b) => b - a); // descending frequency  
  const maxFreq = counts[0];  
  const idleSlots = (maxFreq - 1) * n;  
  let available = counts.slice(1).reduce((s, c) => s + Math.min(c, maxFreq - 1), 0);  
  const idle = Math.max(0, idleSlots - available);  
  return tasks.length + idle;  
}
```

Sorting by remaining frequency descending is what lets the greedy cooldown-filling argument work.

50. Non-overlapping Intervals — max removals to make the rest non-overlapping.

```
function eraseOverlapIntervals(intervals) {  
  intervals.sort((a, b) => a[1] - b[1]); // sort by end time ascending
```



```

let count = 0, prevEnd = -Infinity;
for (const [start, end] of intervals) {
  if (start >= prevEnd) prevEnd = end;
  else count++;
}
return count;
}

```

51. Minimum Number of Arrows to Burst Balloons.

```

function findMinArrowShots(points) {
  points.sort((a, b) => a[1] - b[1]);
  let arrows = 1, end = points[0][1];
  for (const [start, e] of points) {
    if (start > end) { arrows++; end = e; }
  }
  return arrows;
}

```

52. Reorganize String — max-heap by character frequency.

```

function reorganizeString(s) {
  const freq = new Map();
  for (const c of s) freq.set(c, (freq.get(c) || 0) + 1);
  const heap = new MaxHeap((a, b) => a.count - b.count); // by count
  for (const [ch, count] of freq) heap.push({ ch, count });
  let result = "";
  let prev = null;
  while (heap.size() > 0) {
    const top = heap.pop();
    result += top.ch;
    if (prev && prev.count > 0) heap.push(prev);
    top.count--;
    prev = top;
  }
  return result.length === s.length ? result : "";
}

```

53. Wiggle Sort — $\text{nums}[0] < \text{nums}[1] > \text{nums}[2] < \text{nums}[3] \dots$

```

function wiggleSort(nums) {
  for (let i = 0; i < nums.length - 1; i++) {
    const shouldBeLess = i % 2 === 0;
    if ((shouldBeLess && nums[i] > nums[i + 1]) ||
        (!shouldBeLess && nums[i] < nums[i + 1])) {
      [nums[i], nums[i + 1]] = [nums[i + 1], nums[i]];
    }
  }
}

```

A single greedy left-to-right pass with adjacent swaps solves this in $O(n)$; a full sort works too but is unnecessary $O(n \log n)$ overhead.

54. H-Index.

```
function hIndex(citations) {
  citations.sort((a, b) => b - a); // descending
  let h = 0;
  for (let i = 0; i < citations.length; i++) {
    if (citations[i] >= i + 1) h = i + 1;
    else break;
  }
  return h;
}
```

55. Employee Free Time.

```
function employeeFreeTime(schedule) {
  const intervals = schedule.flat().sort((a, b) => a[0] - b[0]);
  const gaps = [];
  let end = intervals[0][1];
  for (const [s, e] of intervals.slice(1)) {
    if (s > end) gaps.push([end, s]);
    end = Math.max(end, e);
  }
  return gaps;
}
```

56. Course Schedule with priority — lexicographically smallest valid ordering.

```
function courseOrder(numCourses, prerequisites) {
  const indegree = new Array(numCourses).fill(0);
  const graph = Array.from({ length: numCourses }, () => []);
  for (const [course, pre] of prerequisites) {
    graph[pre].push(course);
    indegree[course]++;
  }
  const heap = new MinHeap((a, b) => a - b); // min-heap of available course ids
  for (let c = 0; c < numCourses; c++) if (indegree[c] === 0) heap.push(c);
  const order = [];
  while (heap.size() > 0) {
    const c = heap.pop();
    order.push(c);
    for (const next of graph[c]) {
      if (--indegree[next] === 0) heap.push(next);
    }
  }
  return order.length === numCourses ? order : [];
}
```

Using a min-heap instead of a plain queue for the topological sort's frontier is what guarantees the lexicographically smallest valid order.

7. Comparators with Heaps / Priority Queues

57. Max-heap from a min-heap-only implementation, by inverting the comparator.

```
class MinHeap {
  constructor(cmp = (a, b) => a - b) { this.cmp = cmp; this.data = []; }
  size() { return this.data.length; }
  push(val) {
    this.data.push(val);
    let i = this.data.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (this.cmp(this.data[i], this.data[p]) < 0) {
        [this.data[i], this.data[p]] = [this.data[p], this.data[i]]; i = p;
      } else break;
    }
  }
  pop() {
    const top = this.data[0];
    const last = this.data.pop();
    if (this.data.length) { this.data[0] = last; this._sinkDown(0); }
    return top;
  }
  _sinkDown(i) {
    const n = this.data.length;
    while (true) {
      let smallest = i, l = 2*i+1, r = 2*i+2;
      if (l < n && this.cmp(this.data[l], this.data[smallest]) < 0) smallest = l;
      if (r < n && this.cmp(this.data[r], this.data[smallest]) < 0) smallest = r;
      if (smallest === i) break;
      [this.data[i], this.data[smallest]] = [this.data[smallest], this.data[i]];
      i = smallest;
    }
  }
}

// Max-heap: same class, comparator inverted
const maxHeap = new MinHeap((a, b) => b - a);
```

This MinHeap class is reused for the other heap-based answers in this doc; only the comparator passed to the constructor changes.

58. Priority queue that pops lowest priority number first, earliest-inserted breaks ties.

```
let counter = 0;
const pq = new MinHeap((a, b) => a.priority - b.priority || a.seq - b.seq);
function push(item, priority) {
  pq.push({ item, priority, seq: counter++ });
}
```

Encoding an ever-increasing insertion counter as the final tiebreaker is how you fake stability in a heap, which has no inherent ordering guarantee among equal elements.

59. Running median with two heaps.

```
const small = new MinHeap((a, b) => b - a); // max-heap: lower half
const large = new MinHeap((a, b) => a - b); // min-heap: upper half

function addNum(num) {
  small.push(num);
  large.push(small.pop());
  if (large.size() > small.size()) small.push(large.pop());
}

function findMedian() {
  if (small.size() > large.size()) return small.data[0];
  return (small.data[0] + large.data[0]) / 2;
}
```

60. Dijkstra's algorithm — priority queue of (node, distance) by distance ascending.

```
const pq = new MinHeap((a, b) => a.dist - b.dist);
pq.push({ node: source, dist: 0 });
const dist = { [source]: 0 };
while (pq.size() > 0) {
  const { node, dist: d } = pq.pop();
  if (d > (dist[node] ?? Infinity)) continue; // stale entry
  for (const [next, weight] of graph[node]) {
    const nd = d + weight;
    if (nd < (dist[next] ?? Infinity)) {
      dist[next] = nd;
      pq.push({ node: next, dist: nd });
    }
  }
}
```

61. Pop log/event objects in timestamp order via a heap-based comparator.

```
const events = new MinHeap((a, b) => a.timestamp - b.timestamp);
for (const e of rawEvents) events.push(e);
const ordered = [];
while (events.size() > 0) ordered.push(events.pop());
```

62. N-th Ugly Number via min-heap with de-duplication.

```
function nthUglyNumber(n) {
  const heap = new MinHeap((a, b) => a - b);
  const seen = new Set([1]);
  heap.push(1);
  let ugly = 1;
  for (let i = 0; i < n; i++) {
    ugly = heap.pop();
    for (const f of [2, 3, 5]) {
      const next = ugly * f;
      if (!seen.has(next)) { seen.add(next); heap.push(next); }
    }
  }
}
```

```

    }
    return ugly;
  }

```

8. Multi-Criteria & Tricky Tie-Breaking Practice

63. Flights: stops asc, then duration asc, then price asc.

```

flights.sort((a, b) =>
  a.stops - b.stops ||
  a.duration - b.duration ||
  a.price - b.price
);

```

64. File explorer sort: folders before files, then name alphabetically (case-insensitive).

```

entries.sort((a, b) => {
  if (a.isFolder !== b.isFolder) return a.isFolder ? -1 : 1;
  return a.name.localeCompare(b.name, undefined, { sensitivity: 'base' });
});

```

65. Consistent null-handling rule (nulls sort last) without breaking transitivity.

```

function compareNullsLast(cmp) {
  return (a, b) => {
    if (a == null && b == null) return 0;
    if (a == null) return 1;
    if (b == null) return -1;
    return cmp(a, b);
  };
}
values.sort(compareNullsLast((a, b) => a - b));

```

Every pairwise comparison uses the exact same rule for null, so equivalence stays transitive: two nulls are always equal to each other and always greater than any non-null.

66. Locale-aware, case-insensitive sort that preserves original case in the output.

```

strings.sort((a, b) => a.localeCompare(b, undefined, { sensitivity: 'base' }));

```

`localeCompare` with `sensitivity: 'base'` ignores case (and accents) purely for ordering — it does not alter the strings themselves, so original casing is untouched in the output array.

67. A buggy non-transitive comparator, and the fix.

```

// BUGGY: "close enough" tolerance breaks transitivity of equality.
// a=1.0, b=1.05, c=1.1 -> a~b (within .06), b~c (within .06), but a!~c (diff .1)
const buggyCmp = (a, b) => (Math.abs(a - b) < 0.06 ? 0 : a - b);

```

```

// FIX: compare exact values, or bucket into fixed-width bins so equality
// is well-defined and transitive.

```

```
const fixedCmp = (a, b) => a - b;
// or, if binning is required:
const bin = x => Math.round(x / 0.1);
const binnedCmp = (a, b) => bin(a) - bin(b);
```

The general lesson: any tolerance-based "approximately equal" comparator is a transitivity trap — either compare exact values or quantize into fixed, non-overlapping buckets first.

68. isDraft asc, lastModified desc, title asc.

```
docs.sort((a, b) =>
  Number(a.isDraft) - Number(b.isDraft) ||
  new Date(b.lastModified) - new Date(a.lastModified) ||
  a.title.localeCompare(b.title)
);
```

Number(false) = 0 and Number(true) = 1, so 'not draft' (false) naturally sorts before 'draft' (true) under ascending order.

69. Multiples of 3 first (ascending), then all other numbers (ascending).

```
arr.sort((a, b) => {
  const aMult = a % 3 === 0, bMult = b % 3 === 0;
  if (aMult !== bMult) return aMult ? -1 : 1;
  return a - b;
});
```

70. RGB color objects sorted by computed perceived brightness.

```
const brightness = ({ r, g, b }) => 0.299 * r + 0.587 * g + 0.114 * b;
colors.sort((c1, c2) => brightness(c1) - brightness(c2));
```

Per the theory doc's decorate-sort-undecorate note, if brightness() were expensive you'd precompute it once per color instead of recomputing it on every comparison.